

BANK MANAGEMENT SYSTEM

PROJECT REPORT

A Python-Based Command-Line Banking Simulation

Submitted by

Krishna Tripathi

Reg. No.: 26BAI10869

B.Tech CSE AI/ML

VIT Bhopal University

Academic Project

Python 3

Project repository:

https://github.com/KrisHnA-TripAtHi11/Bank_Management_System/tree/main

Table of Contents

1	1. Abstract
2	2. Introduction
3	3. Problem Statement
4	4. Objectives
5	5. Scope of the Project
6	6. Target Users
7	7. Technologies Used
8	8. Functional Requirements
9	9. Non-Functional Requirements
10	10. System Features and Functional Modules
11	11. System Architecture
12	12. Workflow / Process Flow
13	13. Use Case Diagram
14	14. Sequence Diagram
15	15. Component / Module Diagram
16	16. Data / Storage Design
17	17. Working of the System
18	18. Programming Concepts Used
19	19. Design Decisions & Rationale
20	20. Testing and Results
21	21. Output Screenshots
22	22. Limitations
23	23. Future Enhancements
24	24. Challenges Faced
25	25. Learnings & Key Takeaways
26	26. Conclusion
27	27. References

1. Abstract

The Bank Management System is a Python-based academic project that simulates common banking activities through a command-line interface. The system allows a user to create an account, log in using an account number and PIN, check the account balance, deposit money, withdraw money, transfer money, view transaction history, view account details, change the PIN, and log out.

The project demonstrates fundamental Python programming concepts such as functions, dictionaries, loops, conditional statements, user input, input validation, and random number generation. The application is intended only as an educational simulation and does not connect to a real bank, financial institution, database, or payment service.

2. Introduction

Banking applications provide a practical context for demonstrating how programming concepts can be combined to solve a real-world-style problem. This project models a simplified banking environment in which account information and transactions are managed during program execution.

The application uses a text-based menu so that the user can navigate through banking operations. The supplied execution screenshots demonstrate account creation, login, transactions, account information, PIN change, logout, bank information, and program exit.

3. Problem Statement

The project is designed to provide users with a simple banking simulation where they can create a bank account, log in using an account number and PIN, check their balance, deposit money, withdraw money, transfer money between accounts, view transaction history, view account details, and change the PIN.

The system is intended for academic use. Account information is stored temporarily in memory while the Python program is running.

4. Objectives

- Develop a simple banking simulation using Python.
- Demonstrate functions and modular programming through separate banking operations.
- Use dictionaries and lists for account and transaction information.
- Implement basic input validation.
- Generate account numbers and transaction IDs.
- Implement basic banking transactions.
- Provide a simple command-line interface.
- Apply fundamental Python concepts to a practical problem.

5. Scope of the Project

The project focuses on basic banking operations in an academic simulation. It is not intended to replace or connect to a real banking system.

The system does not use a real financial institution, payment gateway, database, or external banking service. Account information is maintained only while the Python program is running.

6. Target Users

The primary target users are students and faculty who need a simple demonstration of basic Python programming concepts through a practical application. The system can also be used as a classroom demonstration of menu-driven programs, functions, dictionaries, loops, validation, and transaction logic.

7. Technologies Used

Python 3: Used to implement the complete application.

random module: Used to generate account numbers and transaction IDs.

Command-Line Interface (CLI): Used for all user interaction through input and printed output.

8. Functional Requirements

ID	Functional requirement
FR1	Account creation with name, mobile number, account type, PIN and initial deposit.
FR2	Login using account number and PIN.
FR3	Check the current account balance.
FR4	Deposit money into the logged-in account.
FR5	Withdraw money when sufficient balance is available.
FR6	Transfer money to another registered account after PIN confirmation.
FR7	View transaction history.
FR8	View account details.
FR9	Change the four-digit PIN.
FR10	Logout and return to the main menu.

9. Non-Functional Requirements

Requirement	How the project addresses it
Usability	Numbered menus and simple text messages make the CLI easy to follow.
Reliability	The program checks input conditions such as valid digits, PIN length, positive amounts and available balance before completing operations.
Maintainability	Individual banking operations are implemented as separate functions, making the basic program easier to understand and modify.
Resource efficiency	The project uses simple in-memory dictionaries and lists and does not require a database or external service.
Validation	User-entered name, mobile number, PIN, account type and transaction amounts are checked before use.
Scope control	The system clearly operates as an academic simulation and does not perform real financial transactions.

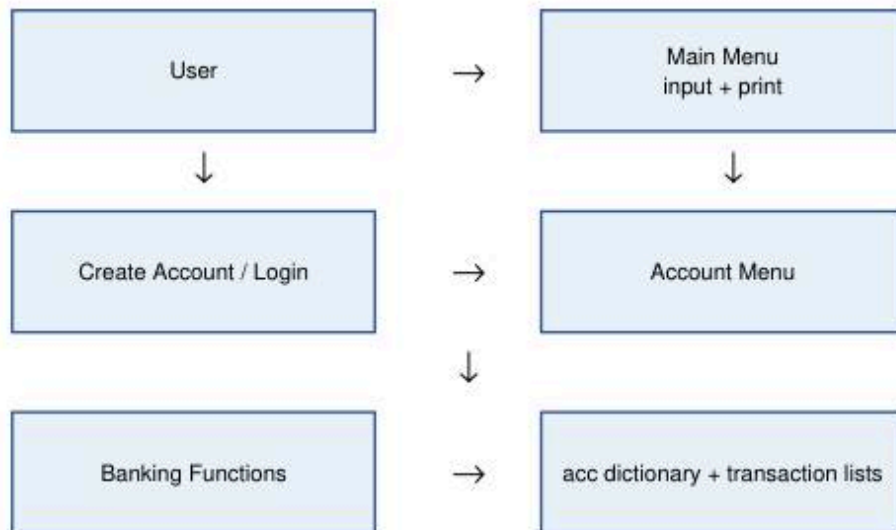
10. System Features and Functional Modules

Functional module	Functions/features included
Module 1 – Account Management	Create New Account, Login, Account Details, Change PIN.
Module 2 – Banking Transactions	Check Balance, Deposit Money, Withdraw Money, Transfer Money.
Module 3 – Transaction & Session Management	Transaction History, Logout, Bank Information and Exit.

The modules above describe the functional organization of the existing single-file basic-Python implementation. No additional advanced programming structures are required to represent these functional modules.

11. System Architecture

The project uses a simple menu-driven architecture. The user interacts with the main menu and, after successful login, the account menu. Each menu choice calls a corresponding Python function. Account information is maintained in the in-memory **acc** dictionary and transaction entries are maintained in each account's transaction list.

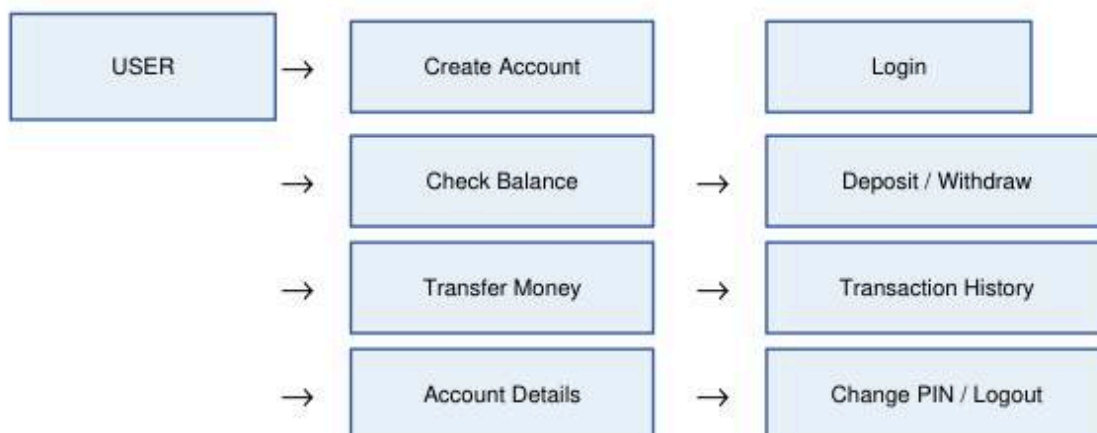


Architecture note: this is a basic Python CLI design. It does not use a database, file handling, external API, or advanced framework.

12. Workflow / Process Flow

1	Start the Python program.
2	Display the main menu.
3	User chooses Create Account, Login, Bank Information, or Exit.
4	For account creation, validate details and create a unique account number.
5	For login, verify account number and PIN.
6	After successful login, display the account menu.
7	User selects balance, deposit, withdrawal, transfer, history, details, or PIN change.
8	Perform the selected operation and update the in-memory account data.
9	User can continue using the account menu or choose Logout.
10	Logout returns to the main menu; Exit ends the program.

13. Use Case Diagram



The single actor is the User. All listed use cases correspond directly to functions/features already present in the supplied Python program.

14. Sequence Diagram

Step	User	PyBank function flow
1	Select Login	login() receives account number and PIN.
2	Enter credentials	Account dictionary is checked.
3	Successful login	account_menu() is opened.
4	Select transaction	deposit(), withdraw(), or transfer() is called.
5	Enter amount/details	Input is validated.
6	Operation succeeds	Balance and transaction list are updated.
7	Display result	Updated balance/transaction ID is printed.

15. Component / Module Diagram

Main program	main_menu()	Controls overall application flow.
Account component	create_acc(), login(), account_details(), change_pin()	Creates and manages account information.
Transaction component	check_balance(), deposit(), withdraw(), transfer()	Performs banking transactions.
History component	transaction_history()	Displays recorded transactions.
Utility component	acc_number(), transaction_id(), valid()	Generates identifiers and validates basic input.

This component view describes the functions already present in the existing single-file program; it does not imply that separate Python class files are used.

16. Data / Storage Design

The project does not use persistent storage. During execution, account data is stored in the global **acc** dictionary. Each account record contains the account holder's name, mobile number, account type, PIN, balance, and a list of transaction messages.

Account dictionary key	Account number
name	Account holder name
mobile	10-digit mobile number
type	Savings or Current
pin	Four-digit PIN used by the academic simulation
balance	Current account balance
transactions	List of account-opening and transaction entries

17. Working of the System

The demonstrated execution begins with the PyBank welcome screen. The user selects account creation and enters the required details. The program confirms successful account creation. The user then logs in with the generated account number and PIN.

In the demonstrated run, the account starts with an initial deposit of Rs. 1000. A deposit of Rs. 500 increases the balance to Rs. 1500. A withdrawal of Rs. 800 reduces the balance to Rs. 700. A transfer of Rs. 600 is then completed, leaving a balance of Rs. 100. The transaction history records the account opening, deposit, withdrawal, and transfer.

The account-details screen displays the account holder information, account number, account type, and current balance. The user then changes the PIN and logs out. Finally, the bank-information screen and exit option are demonstrated.

18. Programming Concepts Used

Functions and modular programming for separating individual banking operations.

Dictionaries for storing account information.

Lists for maintaining transaction entries.

Conditional statements for menu selection and decision-making.

Loops for repeatedly displaying menus and accepting user choices.

User input and validation for account and transaction operations.

Random number generation for account numbers and transaction IDs.

String formatting and console output for presenting results clearly.

19. Design Decisions & Rationale

The project deliberately uses basic Python constructs because the objective is to demonstrate fundamental programming concepts in an understandable academic application.

A dictionary is suitable for holding account records during execution because it provides a simple mapping from account number to account information. Lists are used for transaction history because transactions are displayed in sequence.

A command-line interface keeps the implementation focused on Python logic rather than graphical or web technologies. Random number generation provides simple account numbers and transaction IDs for the simulation.

The project does not introduce persistent storage or advanced security mechanisms because the current scope is a basic academic simulation. These are documented as future enhancements rather than hidden limitations.

20. Testing and Results

Test Case	Input / Action	Observed Result	Status
Account creation	New account with initial deposit Rs. 1000	Account created successfully	Pass
Login	Account number and PIN	Login successful; account menu displayed	Pass
Balance	Select Check Balance	Balance displayed as Rs. 1000 initially	Pass
Deposit	Deposit Rs. 500	New balance Rs. 1500	Pass
Withdrawal	Withdraw Rs. 800	Remaining balance Rs. 700	Pass
Transfer	Transfer Rs. 600 to another account	Transfer successful; remaining balance Rs. 100	Pass
Transaction history	Select Transaction History	Account and transaction entries displayed	Pass
Account details	Select Account Details	Account information and balance displayed	Pass
Change PIN	Replace existing PIN with a new 4-digit PIN	PIN changed successfully	Pass
Logout	Select Logout	Logged out and returned to main menu	Pass
Bank information	Select Bank Information	About PyBank information displayed	Pass
Exit	Select Exit	Closing message displayed	Pass

The testing evidence is based on the supplied execution screenshots and test results in the original report. The screenshots demonstrate the normal successful flow of the main functions.

21. Output Screenshots

The following pages reproduce the execution evidence supplied in the original report. They show account creation, login, balance enquiry, deposit, withdrawal, transfer, transaction history, account details, PIN change, logout, bank information and exit.

11. Output Screenshots

The following screenshots were supplied as evidence of the program execution. They are included in the report to document the working of the implemented modules.

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Jeet Choudhary/bank_management.py =====

=====
WELCOME TO PYBANK
=====
1. Create New Account
2. Login
3. Bank Information
4. Exit
Enter your choice: 1

=====
CREATE NEW ACCOUNT
=====
Enter your full name: Jeet Choudhary
Enter mobile number: 9876543211

Account Type
1. Savings
2. Current
Choose account type: 1
Create a 4-digit PIN: 1234
Confirm PIN: 1234
Enter initial deposit: 1000

Account created successfully!
```

Figure 1. Program startup and new account creation.

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=====
WELCOME TO PYBANK
=====
1. Create New Account
2. Login
3. Bank Information
4. Exit
Enter your choice: 2

=====
LOGIN
=====
Enter account number: 93054051
Enter PIN: 1234

Login successful!
Welcome, Jeet Choudhary

=====
PYBANK ACCOUNT MENU
=====
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Transaction History
6. Account Details
7. Change PIN
8. Logout
```

Figure 2. Login with account number and PIN.

```

C:\MSDOS\11\11\11
File Edit Shell Debug Options Window Help
-----
PYBANK ACCOUNT MENU
-----
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Transaction History
6. Account Details
7. Change PIN
8. Logout

Enter your choice: 1

-----
ACCOUNT BALANCE
-----
Available Balance: Rs. 1000

-----
PYBANK ACCOUNT MENU
-----
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Transaction History
6. Account Details
7. Change PIN
8. Logout

```

Figure 3. Account menu and balance inquiry.

```

-----
PYBANK ACCOUNT MENU
-----
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Transaction History
6. Account Details
7. Change PIN
8. Logout

Enter your choice: 2

-----
DEPOSIT
-----
Enter amount to deposit: 500

Deposit successful!
Amount Deposited: Rs. 500
New Balance: Rs. 1500

```

Figure 4. Deposit operation.

```

PYBANK ACCOUNT MENU
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Transaction History
6. Account Details
7. Change PIN
8. Logout

Enter your choice: 3

WITHDRAW

Enter amount to withdraw: 800

Withdrawal successful!
Amount Withdrawn: Rs. 800
Remaining Balance: Rs. 700

```

Figure 5. Withdrawal operation.

```

PYBANK ACCOUNT MENU
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Transaction History
6. Account Details
7. Change PIN
8. Logout

Enter your choice: 4

MONEY TRANSFER

Enter receiver account number: 22420504
Enter amount to transfer: 600
Enter your PIN to confirm: 1234

Transfer successful!
Transaction ID: TXN890416
Amount Sent: Rs. 600
Receiver: Aditya Sharma
Remaining Balance: Rs. 100

```

Figure 6. Money transfer operation.

```
*IDLE Shell 3.13.14*
File Edit Shell Debug Options Window Help

=====
PYBANK ACCOUNT MENU
=====
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Transaction History
6. Account Details
7. Change PIN
8. Logout

Enter your choice: 5

=====
TRANSACTION HISTORY
=====
1. Account opened with Rs. 1000
2. TXN999427 - Deposited Rs. 500
3. TXN393660 - Withdrawn Rs. 800
4. TXN890416 - Sent Rs. 600 to 22420504
```

Figure 7. Transaction history.

```
=====
PYBANK ACCOUNT MENU
=====
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Transaction History
6. Account Details
7. Change PIN
8. Logout

Enter your choice: 6

=====
ACCOUNT DETAILS
=====
Name       : Jeet Choudhary
Mobile     : 9876543211
Account No. : 83054051
Account Type : Savings
Balance    : Rs. 100
```

Figure 8. Account details.

```
PyQt5 Shell 3.13.14*
File Edit Shell Debug Options Window Help

=====
PYBANK ACCOUNT MENU
=====
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Transaction History
6. Account Details
7. Change PIN
8. Logout

Enter your choice: 7

=====
CHANGE PIN
=====
Enter current PIN: 1234
Enter new 4-digit PIN: 1111
Confirm new PIN: 1111
PIN changed successfully!
```

Figure 9. PIN change operation.

```
=====
PYBANK ACCOUNT MENU
=====
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Transaction History
6. Account Details
7. Change PIN
8. Logout

Enter your choice: 8
Logged out successfully.

=====
WELCOME TO PYBANK
=====
1. Create New Account
2. Login
3. Bank Information
4. Exit

Enter your choice: |
```

Figure 10. Logout and return to main menu.

```
Python Shell 3.10.10
File Edit Shell Debug Options Window Help

=====
WELCOME TO PYBANK
=====
1. Create New Account
2. Login
3. Bank Information
4. Exit

Enter your choice: 3

=====
ABOUT PYBANK
=====
PyBank is a Python-based banking simulation.
It demonstrates basic banking operations
using Python programming concepts.

This is an academic simulation and
does not connect to a real bank.

=====
WELCOME TO PYBANK
=====
1. Create New Account
2. Login
3. Bank Information
4. Exit

Enter your choice: |
```

Figure 11. Bank information screen.

```
=====
WELCOME TO PYBANK
=====
1. Create New Account
2. Login
3. Bank Information
4. Exit

Enter your choice: 4

Thank you for using PyBank!
Have a great day!
>>>
```

Figure 12. Exit message.

12. Limitations

- The system is an academic simulation and does not perform real financial transactions.
- Account information is stored only during program execution and is not connected to a persistent database.
- The application uses a command-line interface rather than a graphical or web interface.
- It is not integrated with real banking services, payment gateways, or external authentication systems.

13. Future Enhancements

- Add persistent storage using a database or structured file format.
- Add a graphical user interface or web interface.
- Introduce stronger authentication and secure PIN handling.

22. Limitations

The system is an academic simulation and does not perform real financial transactions.

Account information is stored only during program execution and is not connected to a persistent database.

The application uses a command-line interface rather than a graphical or web interface.

It is not integrated with real banking services, payment gateways, or external authentication systems.

23. Future Enhancements

Add persistent storage using a database or structured file format.

Add a graphical user interface or web interface.

Introduce stronger authentication and secure PIN handling.

Add richer validation and error-handling scenarios.

Support multiple persistent users and administrative functions.

Add printable or exportable account and transaction reports.

24. Challenges Faced

Designing a complete banking workflow using only basic Python constructs requires careful coordination of dictionaries, lists, functions, loops and conditional statements.

Maintaining correct balances across deposits, withdrawals and transfers is another important challenge. The program checks the required conditions before changing the balance.

Providing a clear menu-driven user experience while keeping the implementation simple is also a key design consideration.

25. Learnings & Key Takeaways

The project provided practical experience with functions, dictionaries, lists, loops, conditional statements, input validation, random number generation and console-based program design.

It also demonstrated how individual Python functions can be combined into a larger application with a clear workflow, functional requirements, testing evidence and project documentation.

26. Conclusion

The Bank Management System successfully demonstrates how fundamental Python programming concepts can be combined to create a practical command-line application. The supplied execution screenshots show successful account creation, login, balance checking, deposit, withdrawal, transfer, transaction history, account details, PIN change, logout, bank information, and exit operations.

As an academic project, the system provides a clear example of modular programming, data handling with dictionaries, input processing, validation, and transaction-oriented logic. It also provides a foundation that could be extended with persistent storage, a graphical interface, and stronger security in future versions.

27. References

Project Statement supplied for the Bank Management System.

Program execution screenshots supplied with the project.

VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines.

Source code repository: <https://github.com/jeetjat204-cpu/Bank-Management-System>